

Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria de Ingeniería y Ciencias Sociales y Administrativas



REPORTE DE PRÁCTICAS

Integrantes:

- Gómez Tavarez Edder Giovani
- Hernandez Contreras David Antonio
- Peláez Ruíz Monserrat

Secuencia: 3CM31
Unidad de Aprendizaje: Abstracción y Uso de Datos
Profesor: Entzana Garduño Yventz

Liga GitHub: https://github.com/DarkExeX/Abstraccion.git

ÍNDICE

Reporte de prácticas · Abstracción y Uso de Datos

PRIMER DEPARTAMENTAL

Práctica 1	Hola Mundo	3
Práctica 2	Suma de 2 números	3
Práctica 3	Calculadora básica	4
Práctica 4	Calculadora con sobrecarga	6
Práctica 5	Calculadora con herencia	7
Práctica 6	Calculadora con herencia y sobreescritura	8
Práctica 7	Creación de dato	10
Práctica 7 bis	Identificar tamaños	11
Práctica 8	Promedio	13
Práctica 9	Promedio con arreglo	13
Práctica 10	Promedio con puntero al arreglo	14
Práctica 11	Matrices	15
Práctica 12	Nuevo tipo de dato como arreglo	16
Práctica 13	Nuevo tipo de dato con puntero al arreglo	18
Práctica 14	Recursividad: factorial, Fibonacci y calculadora recursiva	19
Práctica 15	Recursividad: fractales (Sierpinski y polvo de Cantor)	24

SEGUNDO DEPARTAMENTAL

Práctica 2	Tipos de dato abstractos (ADT)	27
Práctica 3	Pila estática mediante un arreglo	28
Práctica 4	Cola estática mediante un arreglo	29
Práctica 5	Lista estática mediante un arreglo	30
Práctica 6	Pila dinámica mediante apuntadores	31
Práctica 7	Cola dinámica mediante apuntadores	32
Práctica 7 bis	Lista dinámica mediante apuntadores	34
Práctica 8	Pila dinámica con librerías del IDE (stdlib)	35
Práctica 9	Cola dinámica con librerías del IDE (stdlib)	36
Práctica 10	Lista dinámica con librerías del IDE (stdlib)	37

Conclusiones 40

PRIMER DEPARTAMENTAL

Este departamental reúne las prácticas de fundamentos de programación, sobrecarga, herencia y sobreescritura, creación de tipos de dato, arreglos, punteros, matrices y recursividad. Los programas se desarrollaron en Java, C y C++, según la práctica.

PRÁCTICA 1 · PRIMER DEPARTAMENTAL

Hola Mundo

ERRORES

Durante la elaboración de este programa no se presentó ningún error, ya que se trataba de un ejercicio introductorio para conocer la estructura básica de un programa en Java. La sintaxis se escribió de manera correcta, se declaró la clase y el método principal main, por lo que el programa pudo ejecutarse sin ningún problema y mostrar el mensaje solicitado en pantalla.

SOLUCIÓN DE ERRORES

Debido a que no se presentaron errores durante la elaboración del programa, no fue necesario realizar correcciones en el código. Solamente se verificó que la estructura estuviera escrita de manera correcta y que, al ejecutarlo, se mostrara el mensaje “Hola mundo” en la consola.

CÓDIGO

```
public class HolaMundo{
    public static void main(String[] args) {
        System.out.println("Hola mundo");
    }
}
```

PRÁCTICA 2 · PRIMER DEPARTAMENTAL

Suma de 2 números

ERRORES

Durante la elaboración de este programa no se presentaron errores de sintaxis ni de funcionamiento en la operación de suma, ya que el método suma realizaba correctamente la adición de los dos números recibidos. Sin embargo, uno de los aspectos que se tuvo que mejorar fue que el programa no era interactivo, debido a que los valores de los números se encontraban escritos directamente en el código. Esto ocasionaba que cada vez que se ejecutara el programa se realizara la suma de los mismos valores, sin darle la oportunidad al usuario de ingresar cantidades diferentes.

SOLUCIÓN DE ERRORES

Para solucionar este inconveniente fue necesario modificar el programa e implementar la clase Scanner, la cual permite recibir información que el usuario introduce a través del teclado. También se agregó la declaración de

dos variables para almacenar los números ingresados y posteriormente enviarlos como argumentos al método suma.

De esta manera, se conservó el método que realizaba la operación, pero se modificó la forma de obtener los datos. Con este cambio, el programa dejó de utilizar valores establecidos directamente en el código y permitió que el usuario pudiera realizar diferentes sumas sin necesidad de modificar el programa cada vez.

CÓDIGO

```
import java.util.Scanner;
public class Suma2Numeros {
    public static int suma(int num1, int num2) {
        return num1 + num2;
    }
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        System.out.println("Ingresa el primer numero:");
        int num1 = entrada.nextInt();
        System.out.println("Ingresa el segundo numero:");
        int num2 = entrada.nextInt();
        int resultado = suma(num1, num2);
        System.out.println("El resultado de la suma es: " + resultado);
    }
}
```

PRÁCTICA 3 · PRIMER DEPARTAMENTAL

Calculadora básica

ERRORES

Durante la elaboración de este programa se implementaron las operaciones básicas de suma, resta, multiplicación y división mediante diferentes métodos, utilizando dos números que eran ingresados por el usuario. Sin embargo, al realizar las pruebas, se identificó que en la operación de división no se había considerado qué sucedería si el segundo número ingresado era cero, lo cual podía provocar un resultado incorrecto o un problema durante la ejecución de la operación.

Otro aspecto que se tuvo que tomar en cuenta fue el tipo de resultado que podía obtenerse al realizar una división, ya que no todas las divisiones dan como resultado un número entero. Por ejemplo, al dividir 5 entre 2, el resultado es 2.5, por lo que era necesario permitir que la calculadora pudiera mostrar cantidades con decimales y no solamente números enteros.

Estos aspectos fueron importantes porque, aunque las operaciones principales ya estaban implementadas, todavía era necesario considerar algunas situaciones que podían afectar los resultados y el funcionamiento del programa.

SOLUCIÓN DE ERRORES

Para solucionar el problema de la división entre cero, se agregó una condición if-else dentro del método encargado de realizar esta operación. De esta manera, antes de efectuar la división, el programa comprueba si el segundo número es igual a cero. Si se cumple esta condición, muestra un mensaje indicando que no es posible realizar la operación; de lo contrario, continúa con la división y presenta el resultado. Con esto se evitó que el programa realizara una operación que no está definida matemáticamente.

Por otra parte, para solucionar el problema relacionado con los resultados decimales, se decidió utilizar el tipo de dato float en lugar de int, tanto para almacenar los números ingresados por el usuario como para realizar las operaciones. Esto permitió trabajar con cantidades que tienen parte decimal y obtener resultados más precisos en las operaciones de la calculadora. También se utilizó System.out.printf() para mostrar los resultados con dos cifras decimales, facilitando su lectura.

CÓDIGO ESENCIAL

1. Declaración de la clase y atributos

```
public class Calculadora {
    private float num1;
    private float num2;
}
```

2. Constructor

```
public Calculadora(float num1, float num2) {
    this.num1 = num1;
    this.num2 = num2;
}
```

3. Métodos para las operaciones

```
public float sumar() {
    return num1 + num2;
}
public float restar() {
    return num1 - num2;
}
public float multiplicar() {
    return num1 * num2;
}
```

4. Método para la división

```
public void dividir() {
    if (num2 == 0) {
        System.out.println("No se puede dividir entre cero.");
    } else {
        System.out.printf("La división es: %.2f%n", num1 / num2);
    }
}
```

5. Selección de operaciones a través de switch

```
public void ejecutarOperacion(int opcion) {
    switch (opcion) {
        case 1:
            System.out.printf("La suma es: %.2f%n", sumar());
            break;
        case 2:
            System.out.printf("La resta es: %.2f%n", restar());
            break;
        case 3:
            System.out.printf("La multiplicación es: %.2f%n", multiplicar());
            break;
        case 4:
```

```
            dividir();
            break;
        default:
            System.out.println("Operación no válida.");
    }
}
```

PRÁCTICA 4 · PRIMER DEPARTAMENTAL

Calculadora con sobrecarga

ERRORES

Durante la elaboración de este programa se presentó un problema relacionado con la operación de división, ya que inicialmente no se había considerado qué sucedería si alguno de los números utilizados como divisor tenía un valor de cero. Esto era importante porque, al realizar una división con cero como divisor, se podía obtener un resultado incorrecto o generar un problema en el funcionamiento esperado del programa.

Este inconveniente se presentó principalmente en los métodos dividir, ya que se tenían diferentes versiones dependiendo de la cantidad de parámetros que recibían. En el caso de la división con dos parámetros, era necesario comprobar que el segundo número fuera diferente de cero, mientras que en la división con tres parámetros se debía comprobar tanto el segundo como el tercer número, debido a que ambos se utilizan como divisores.

SOLUCIÓN DE ERRORES

Para solucionar este problema se agregó una condición if en los métodos encargados de realizar la división. En el método que recibe dos parámetros se verificó que el segundo número fuera diferente de cero antes de realizar la operación. Por otra parte, en el método que recibe tres parámetros se agregó una condición para comprobar que tanto el segundo como el tercer número fueran diferentes de cero.

De esta manera, cuando alguno de los divisores tiene un valor de cero, el programa muestra un mensaje indicando que no es posible realizar la división y evita continuar con la operación. En caso contrario, realiza el cálculo y devuelve el resultado correspondiente.

Además de esta corrección, se mantuvieron las diferentes versiones de los métodos para demostrar la sobrecarga, permitiendo realizar las operaciones de suma, resta, multiplicación y división sin parámetros, con dos parámetros y con tres parámetros.

Con esta modificación se consiguió que la calculadora contemplara una situación que inicialmente no se había considerado y que las operaciones de división tuvieran una validación antes de ejecutarse.

CÓDIGO ESENCIAL

1. Sobrecarga del método

```
int sumar() {
    return 0;
}
int sumar(int num1, int num2) {
    return num1 + num2;
}
int sumar(int num1, int num2, int num3) {
    return num1 + num2 + num3;
}
```

2. División con 2 parámetros

```
float dividir(float num1, float num2) {
    if (num2 == 0) {
        System.out.println("No se puede dividir entre 0");
        return 0;
    }
    return num1 / num2;
}
```

3. División con 3 parámetros

```
float dividir(float num1, float num2, float num3) {
    if (num2 == 0 || num3 == 0) {
        System.out.println("No se puede dividir entre 0");
        return 0;
    }
    return num1 / num2 / num3;
}
```

4. Creación del objeto

```
Calculadora calculadora = new Calculadora();
```

PRÁCTICA 5 · PRIMER DEPARTAMENTAL

Calculadora con herencia

ERRORES

Durante la realización de esta práctica no se presentaron errores en el código, ya que se retomó la calculadora realizada anteriormente y sólo se aplicó el concepto de herencia. Se creó una nueva clase llamada CalculadoraNueva, la cual heredó los métodos de la clase Calculadora, por lo que no fue necesario volver a escribir las operaciones que ya estaban programadas.

SOLUCIÓN DE ERRORES

No fue necesario realizar correcciones, debido a que el programa funcionó correctamente. Solamente se tuvo que establecer la relación de herencia entre las clases mediante la palabra reservada extends, lo que permitió que la nueva calculadora pudiera utilizar los métodos de la clase original.

CÓDIGO ESENCIAL

1. Clase padre

```
static class Calculadora {
}
```

2. Creación de la clase hija

```
static class CalculadoraNueva extends Calculadora {
}
```

3. Creación del objeto

```
CalculadoraNueva calculadora = new CalculadoraNueva();
```

4. Uso de los métodos heredados

```
System.out.println("El resultado de la suma es: " + calculadora.sumar(5, 3));
```

PRÁCTICA 6 · PRIMER DEPARTAMENTAL

Calculadora con herencia y sobreescritura

ERRORES

Durante la realización de esta práctica, uno de los problemas que se presentó fue al implementar la multiplicación por medio de sumas sucesivas, ya que inicialmente no se estaba considerando correctamente la cantidad de veces que se debía sumar el primer número para obtener el resultado de la multiplicación. También fue necesario tomar en cuenta los números negativos, porque el ciclo debía realizar las sumas de acuerdo con el valor del segundo número y no solamente cuando este fuera positivo.

Otro problema se presentó al realizar la división por medio de restas sucesivas, debido a que al principio no se estaba considerando correctamente la condición para repetir las restas. Era necesario restar el divisor al dividendo hasta que el resultado fuera menor que el divisor, aumentando el cociente en cada repetición. Además, se tenía que contemplar que no se puede realizar una división entre cero y que el procedimiento implementado solamente trabajaba con números positivos.

SOLUCIÓN DE ERRORES

Para solucionar los problemas de la multiplicación, se utilizó un ciclo for, el cual permite repetir la suma del primer número la cantidad de veces indicada por el segundo número. También se agregó una condición para considerar cuando el segundo número fuera negativo y así obtener el resultado con el signo correspondiente.

En el caso de la división, se utilizó un ciclo while, que permite realizar las restas sucesivas mientras el dividendo sea mayor o igual que el divisor. En cada repetición se resta el divisor al dividendo y se aumenta el valor del cociente. También se agregaron condiciones para evitar la división entre cero y para indicar que solamente se pueden ingresar números positivos en este procedimiento.

CÓDIGO ESENCIAL

1. Herencia de la clase padre

```
static class CalculadoraNueva extends Calculadora {
```

2. Multiplicación por sumas sucesivas

```
@Override
int multiplicar(int num1, int num2) {
    int resultado = 0;
    for (int i = 0; i < Math.abs(num2); i++) {
        resultado = resultado + num1;
    }
    if (num2 < 0) {
        resultado = -resultado;
    }
    return resultado;
}
```

3. Potencia por multiplicaciones sucesivas

```
int potencia(int base, int exponente) {
    int resultado = 1;
    for (int i = 0; i < exponente; i++) {
        resultado = resultado * base;
    }
    return resultado;
}
```

4. División por restas sucesivas

```
@Override
float dividir(float num1, float num2) {
    int cociente = 0;
    if (num2 == 0) {
        System.out.println("No se puede dividir entre 0");
        return 0;
    }
    if (num1 < 0 || num2 < 0) {
        System.out.println("La division solo se puede realizar con numeros positivos");
        return 0;
    }
    while (num1 >= num2) {
        num1 = num1 - num2;
        cociente++;
    }
    return cociente;
}
```

5. Creación del objeto de la clase hija

```
CalculadoraNueva calculadora = new CalculadoraNueva();
```

PRÁCTICA 7 · PRIMER DEPARTAMENTAL

Creación de dato

PROGRAMACIÓN ESTRUCTURADA

ERRORES

Durante la realización de esta práctica no se presentaron errores en el código, ya que se utilizaron correctamente las estructuras struct para crear los nuevos tipos de datos. Se declararon las estructuras Auto y Persona, agregando los atributos que solicitaba la práctica y utilizando los tipos de datos correspondientes para almacenar cada uno de ellos.

SOLUCIÓN DE ERRORES

No fue necesario realizar correcciones, debido a que las estructuras se declararon correctamente y el programa compiló sin presentar problemas. Solamente se tuvo que identificar qué tipo de dato correspondía a cada atributo, utilizando float e int para los datos del automóvil, y arreglos de caracteres, char e int para la información de la persona.

CÓDIGO ESENCIAL

1. Creación del tipo de dato Auto

```
struct Auto {
    float precio;
    int anio;
};
```

2. Creación del tipo de dato Persona

```
struct Persona {
    char nombre[30];
    char ap[30];
    char am[30];
    char  genero;
    int edad;
};
```

PROGRAMACIÓN ORIENTADA A OBJETOS

ERRORES

Durante la realización de esta práctica no se presentaron errores en el código, ya que se utilizaron correctamente las clases para crear los nuevos tipos de datos. Se declararon las clases Auto y Persona, agregando dentro de ellas los atributos que se solicitaron. También se identificaron los tipos de datos que correspondían a cada atributo para poder representar la información de manera adecuada.

SOLUCIÓN DE ERRORES

No fue necesario realizar correcciones, debido a que las clases y sus atributos se declararon correctamente. Se utilizaron los tipos de datos float, int, String y char, dependiendo de la información que se necesitaba almacenar. De esta manera, se pudo representar tanto la información de un automóvil como los datos personales de una persona mediante la creación de clases.

CÓDIGO ESENCIAL

1. Creación de la clase Auto

```
static class Auto {
    float precio;
    int anio;
}
```

2. Creación de la clase Persona

```
static class Persona {
    String nombre;
    String ap;
    String am;
    char genero;
    int edad;
}
```

PRÁCTICA 7 BIS · PRIMER DEPARTAMENTAL

Identificar tamaños

PROGRAMACIÓN ESTRUCTURADA

ERRORES

Durante la realización de esta práctica no se presentaron errores que impidieran el funcionamiento del programa. Sin embargo, fue necesario prestar atención a la forma en que se obtenían los tamaños de los atributos de cada estructura, ya que se estaba utilizando sizeof() para identificar cuántos bytes ocupa cada dato. También fue importante distinguir entre el tamaño de un atributo individual y el tamaño total de una estructura, debido a que este último puede incluir espacios adicionales que agrega el compilador para organizar los datos en memoria.

SOLUCIÓN DE ERRORES

No fue necesario realizar modificaciones importantes en el funcionamiento del programa, solamente se revisaron las instrucciones utilizadas para obtener los tamaños de los datos. Se utilizó sizeof() para conocer el espacio que ocupa cada atributo y también para obtener el tamaño completo de las estructuras Auto y Persona. Además, se identificó que el tipo de dato char ocupa 1 byte en C y que los arreglos de caracteres ocupan el espacio correspondiente a la cantidad de elementos que contienen.

CÓDIGO ESENCIAL

1. Tamaño de la estructura Auto

```
printf("Auto: %zu bytes\n", sizeof(struct Auto));
printf("float precio: %zu bytes\n", sizeof(auto1.precio));
printf("int anio: %zu bytes\n", sizeof(auto1.anio));
```

2. Tamaño de la estructura Persona

```
printf("Persona: %zu bytes\n", sizeof(struct Persona));
printf("nombre: %zu bytes\n", sizeof(persona1.nombre));
printf("ap: %zu bytes\n", sizeof(persona1.ap));
printf("am: %zu bytes\n", sizeof(persona1.am));
printf("genero: %zu bytes\n", sizeof(persona1.genero));
printf("edad: %zu bytes\n", sizeof(persona1.edad));
```

PROGRAMACIÓN ORIENTADA A OBJETOS

ERRORES

Durante la realización de esta práctica se presentó una confusión al momento de calcular el tamaño de la clase Persona, ya que inicialmente se estaba sumando la cantidad de elementos de los arreglos de caracteres sin considerar el espacio que ocupa cada uno de ellos. Esto provocaba que el resultado del tamaño de la persona no fuera correcto, porque en Java cada elemento de tipo char ocupa 2 bytes. También fue necesario identificar que los tamaños de los tipos de datos en Java no son iguales a los que se utilizan en C, como sucede con el tipo de dato char.

SOLUCIÓN DE ERRORES

Para solucionar este problema se modificó la operación que calculaba el tamaño de la clase Persona, multiplicando por 2 la cantidad de elementos de cada arreglo de caracteres. También se utilizaron las propiedades Float.BYTES, Integer.BYTES y Character.BYTES para obtener el tamaño de los tipos de datos básicos. De esta manera, se pudieron identificar los tamaños de los atributos de las clases Auto y Persona, tomando en cuenta las características de Java.

CÓDIGO ESENCIAL

1. Tamaño de los atributos de Auto

```
System.out.println("float precio: " + Float.BYTES + " bytes");
System.out.println("int anio: " + Integer.BYTES + " bytes");
int tamAuto = Float.BYTES + Integer.BYTES;
System.out.println("Auto: " + tamAuto + " bytes");
```

2. Tamaño de los atributos de Persona

```
System.out.println("nombre: " + persona.nombre.length * 2 + " bytes");
System.out.println("ap: " + persona.ap.length * 2 + " bytes");
System.out.println("am: " + persona.am.length * 2 + " bytes");
System.out.println("char genero: " + Character.BYTES + " bytes");
System.out.println("int edad: " + Integer.BYTES + " bytes");
```

3. Cálculo del tamaño de Persona

```
int tamPersona = (persona.nombre.length * 2)
    + (persona.ap.length * 2)
    + (persona.am.length * 2)
    + Character.BYTES
    + Integer.BYTES;
System.out.println("Persona: " + tamPersona + " bytes");
```

PRÁCTICA 8 · PRIMER DEPARTAMENTAL

Promedio

ERRORES

Al realizar el programa para calcular el promedio de cinco calificaciones, inicialmente no se consideró que el resultado de la división podía contener decimales. Esto podía ocasionar que el promedio se mostrara como un número entero y se perdiera la parte decimal del resultado. Por ejemplo, si la suma de las cinco calificaciones no era divisible exactamente entre cinco, el resultado podía ser incorrecto si se realizaba una división entera.

SOLUCIÓN DE ERRORES

Para solucionar este problema, se modificó la operación de la división para que se realizara utilizando un valor decimal. En Java se utilizó 5.0f en lugar de 5, de esta manera el resultado de la operación se guarda correctamente en la variable de tipo float.

Con este cambio, el programa puede mostrar promedios que contienen decimales, sin perder información durante la operación. También se mantuvo el uso de float para almacenar el promedio.

CÓDIGO ESENCIAL

1. Cálculo del promedio

```
suma = cal1 + cal2 + cal3 + cal4 + cal5;
promedio = suma / 5.0f;
```

2. Declaración de la variable

```
float promedio;
```

3. Impresión del resultado

```
System.out.println("El promedio es: " + promedio);
```

PRÁCTICA 9 · PRIMER DEPARTAMENTAL

Promedio con arreglo

ERRORES

Al realizar el programa utilizando un arreglo para almacenar las cinco calificaciones, uno de los errores que se presentó fue al momento de utilizar el ciclo for, ya que inicialmente se había colocado que la variable i debía ser igual a 5 para controlar las repeticiones. Sin embargo, esto ocasionaba que el ciclo no funcionara como se esperaba, debido a que el arreglo comienza a contar sus posiciones desde el número 0 y termina en la posición 4. Por lo tanto, al establecer incorrectamente la condición del ciclo, no se recorrían las cinco posiciones del arreglo de la manera adecuada, lo que afectaba tanto el ingreso de las calificaciones como las operaciones que se realizaban posteriormente.

SOLUCIÓN DE ERRORES

Para solucionar este problema, se corrigió la condición del ciclo for, estableciendo que la variable i debía comenzar en 0 y repetirse mientras fuera menor que 5.

De esta manera, el ciclo puede recorrer todas las posiciones del arreglo, desde la posición 0 hasta la posición 4, permitiendo ingresar y trabajar correctamente con las cinco calificaciones.

También se utilizó el mismo procedimiento para recorrer el arreglo y realizar las operaciones de suma, promedio, máximo y mínimo.

CÓDIGO ESENCIAL

1. Declaración del arreglo

```
int[] calificaciones = new int[5];
```

2. Ciclo para ingresar las calificaciones

```
for (int i = 0; i < 5; i++) {
    System.out.print("Calificacion " + (i + 1) + ": ");
    calificaciones[i] = entrada.nextInt();
}
```

3. Ciclo para realizar las operaciones

```
for (int i = 0; i < 5; i++) {
    suma = suma + calificaciones[i];
    if (calificaciones[i] > maximo) {
        maximo = calificaciones[i];
    }
    if (calificaciones[i] < minimo) {
        minimo = calificaciones[i];
    }
}
```

4. Cálculo del promedio

```
promedio = suma / 5.0f;
```

PRÁCTICA 10 · PRIMER DEPARTAMENTAL

Promedio con puntero al arreglo

ERRORES

Al realizar el programa de promedio utilizando un puntero al arreglo, se presentaron algunos detalles en la forma en que se estaban realizando las operaciones. Uno de ellos fue que, al buscar el valor mínimo, se estaba utilizando cal[i] para acceder a los elementos del arreglo, en lugar de utilizar el puntero como se estaba haciendo con las demás operaciones.

Otro detalle fue que el cálculo del promedio se encontraba dentro del ciclo for, por lo que se realizaba en cada repetición, aunque realmente no era necesario. Esto hacía que la operación se repitiera varias veces cuando únicamente se necesitaba realizar una vez después de obtener la suma de las cinco calificaciones.

SOLUCIÓN DE ERRORES

Para solucionar estos detalles, se modificó la condición para encontrar el número mínimo, cambiando cal[i] por *(p + i), de esta manera todas las operaciones se realizan mediante el puntero y se mantiene la misma forma de acceder indirectamente a los elementos del arreglo.

También se cambió la ubicación de la operación del promedio, colocándola después del ciclo for, ya que primero se necesita obtener la suma de las cinco calificaciones y posteriormente realizar la división entre cinco.

Con estas modificaciones, el programa puede realizar correctamente las operaciones solicitadas utilizando el puntero para acceder a los datos del arreglo.

CÓDIGO ESENCIAL

1. Declaración del arreglo y del puntero

```
int cal[5];
int *p;
```

2. Asignación del puntero

```
p = cal;
```

3. Acceso a los elementos mediante el puntero

```
for(int i = 0; i < 5; i++) {
    suma = suma + *(p + i);
    if(*(p + i) > maximo) {
        maximo = *(p + i);
    }
    if(*(p + i) < minimo) {
        minimo = *(p + i);
    }
}
```

4. Cálculo del promedio

```
promedio = suma / 5.0;
```

PRÁCTICA 11 · PRIMER DEPARTAMENTAL

Matrices

ERRORES

Al realizar el programa para multiplicar matrices, uno de los principales errores se presentó al momento de utilizar los ciclos for para realizar la multiplicación de la matriz A por la matriz B. Inicialmente se tuvo dificultad para establecer correctamente los ciclos que se necesitaban y la forma en que debían recorrerse las filas y columnas de ambas matrices. Esto ocasionaba que no se realizaran correctamente las multiplicaciones y sumas necesarias para obtener cada elemento de la matriz resultante. También fue necesario comprender cómo debía funcionar el tercer ciclo for, ya que este permite recorrer los elementos correspondientes de las filas de A y las columnas de B.

SOLUCIÓN DE ERRORES

Para solucionar estos errores, se corrigió la estructura de los ciclos for, utilizando tres ciclos anidados para realizar la multiplicación de las matrices.

El primer ciclo permite recorrer las filas de la matriz A, el segundo recorre las columnas de la matriz B y el tercero realiza las multiplicaciones de los elementos correspondientes para después sumarlos y obtener el resultado de cada posición.

De esta manera, se logró que el programa realizara correctamente la multiplicación de las matrices y almacenara los resultados en una tercera matriz llamada resultado.

CÓDIGO ESENCIAL

1. Declaración de las matrices

```
int[][] A = new int[n][n];
int[][] B = new int[n][n];
int[][] resultado = new int[n][n];
```

2. Multiplicación de las matrices A y B

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        resultado[i][j] = 0;
        for (int k = 0; k < n; k++) {
            resultado[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

3. Impresión de la matriz resultante

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(resultado[i][j] + "\t");
    }
    System.out.println();
}
```

PRÁCTICA 12 · PRIMER DEPARTAMENTAL

Nuevo tipo de dato como arreglo

PROGRAMACIÓN ESTRUCTURADA

ERRORES

Al realizar el programa, una de las dificultades que se presentó fue al momento de guardar los nombres y apellidos de las personas dentro de las estructuras. Inicialmente hubo confusión sobre cómo asignar los textos a los arreglos de caracteres, ya que no se podían colocar directamente utilizando el signo de igual como se hace con otros tipos de datos.

Otro detalle fue al momento de recorrer los arreglos para mostrar la información de las personas y los autos, ya que era necesario utilizar correctamente los índices para acceder a cada elemento y a sus respectivos atributos.

SOLUCIÓN DE ERRORES

Para solucionar el primer problema, se utilizó la función strcpy(), que permite copiar las cadenas de texto dentro de los atributos correspondientes de la estructura Persona.

También se revisaron los ciclos for para que recorrieran las cuatro posiciones de cada arreglo, comenzando desde el índice 0 y terminando en el índice 3. De esta manera se logró mostrar correctamente la información de todas las personas y los autos.

CÓDIGO ESENCIAL

1. Declaración de las estructuras

```
struct Auto {
    float precio;
    int anio;
};
struct Persona {
    char nombre[30];
    char ap[30];
    char am[30];
    char  genero;
    int edad;
};
```

2. Creación de los arreglos

```
struct Persona personas[4];
struct Auto autos[4];
```

3. Asignación de nombres

```
strcpy(personas[0].nombre, "Juan");
strcpy(personas[0].ap, "Martinez");
strcpy(personas[0].am, "Ruiz");
```

4. Recorrido de los arreglos

```
for(int i = 0; i < 4; i++) {
    printf("Persona %d\n", i + 1);
    printf("Nombre: %s\n", personas[i].nombre);
}
```

PROGRAMACIÓN ORIENTADA A OBJETOS

ERRORES

Al realizar el programa en Java, una de las dificultades fue al momento de crear los arreglos de objetos, ya que inicialmente hubo confusión sobre cómo almacenar las personas y los autos dentro de ellos. Aunque se había declarado el tamaño de los arreglos, todavía era necesario crear cada objeto individualmente para poder asignar sus atributos.

Otra dificultad se presentó al momento de mostrar la información mediante los ciclos for, ya que era necesario acceder correctamente a los atributos de cada objeto utilizando su posición dentro del arreglo.

SOLUCIÓN DE ERRORES

Para solucionar el primer problema, se crearon los objetos utilizando la palabra reservada new en cada una de las posiciones de los arreglos. De esta manera fue posible asignar los valores correspondientes a los atributos de cada persona y auto.

También se revisó la forma de recorrer los arreglos y acceder a los atributos mediante el nombre del arreglo, su índice y el nombre del atributo. Con esto se logró mostrar correctamente toda la información almacenada.

CÓDIGO ESENCIAL

1. Declaración de las clases

```
static class Auto {
    float precio;
    int anio;
}
static class Persona {
    String nombre;
    String ap;
    String am;
    char genero;
    int edad;
}
```

2. Creación de los arreglos de objetos

```
Persona[] personas = new Persona[4];
personas[0] = new Persona();
personas[1] = new Persona();
personas[2] = new Persona();
personas[3] = new Persona();
```

3. Asignación de valores

```
personas[0].nombre = "Juan";
personas[0].ap = "Martinez";
personas[0].am = "Ruiz";
personas[0].genero = 'M';
personas[0].edad = 25;
```

4. Recorrido de los arreglos

```
for (int i = 0; i < 4; i++) {
    System.out.println("Persona " + (i + 1));
    System.out.println("Nombre: " + personas[i].nombre);
    System.out.println("Edad: " + personas[i].edad);
}
```

PRÁCTICA 13 · PRIMER DEPARTAMENTAL

Nuevo tipo de dato con puntero al arreglo

ERRORES

Al realizar el programa para acceder de manera indirecta a la lista de personas mediante un puntero al arreglo, una de las dificultades que se presentó fue al momento de asignar la dirección del arreglo al puntero, ya que inicialmente hubo confusión sobre cómo relacionar el puntero con las posiciones de la estructura Persona.

Otro error fue al momento de acceder a los atributos de cada persona utilizando el puntero, debido a que no se tenía completamente claro cómo utilizar el operador -> para obtener los datos de cada estructura. Esto ocasionaba confusión al momento de recorrer el arreglo y mostrar la información correspondiente.

SOLUCIÓN DE ERRORES

Para solucionar estos problemas, se revisó la forma en que se asigna la dirección del primer elemento del arreglo al puntero, utilizando la instrucción p = personas;. De esta manera, el puntero puede utilizarse para recorrer las diferentes posiciones del arreglo sin necesidad de acceder directamente mediante el nombre de este.

También se utilizó correctamente el operador ->, que permite acceder a los atributos de cada estructura mediante el puntero. Además, se mantuvo el ciclo for para recorrer las cuatro posiciones del arreglo, utilizando p + i para desplazarse entre las personas y mostrar sus datos.

Con estas modificaciones se logró acceder correctamente a la información de cada persona de manera indirecta.

CÓDIGO ESENCIAL

1. Declaración de la estructura y del puntero

```
struct Persona {
    char nombre[30];
    char ap[30];
    char am[30];
    char  genero;
    int edad;
};
struct Persona personas[4];
struct Persona *p;
```

2. Asignación del puntero

```
p = personas;
```

3. Acceso indirecto a los atributos

```
for (int i = 0; i < 4; i++) {
    printf("Nombre: %s\n", (p + i)->nombre);
    printf("Apellido paterno: %s\n", (p + i)->ap);
    printf("Apellido materno: %s\n", (p + i)->am);
    printf("Genero: %c\n", (p + i)->genero);
    printf("Edad: %d\n", (p + i)->edad);
}
```

PRÁCTICA 14 · PRIMER DEPARTAMENTAL

Recursividad: factorial, Fibonacci y calculadora recursiva

ERRORES

Factorial. Al probar el programa con valores grandes se observó que el resultado dejaba de ser correcto. Aunque se utilizó el tipo unsigned long long, el factorial de 21 ya no cabe en 64 bits, por lo que el programa mostraba 14197454024290336768 en lugar de 51090942171709440000 sin marcar ningún error. Además, si el usuario escribía una letra en lugar de un número, la lectura con cin fallaba y el programa mostraba que el factorial de 0 es 1, un resultado incorrecto para esa entrada.

Fibonacci. El principal problema fue la eficiencia. La versión recursiva calcula muchas veces los mismos valores (por ejemplo, para obtener fibonacci(5) se calcula más de una vez fibonacci(3) y fibonacci(2)), por lo que el tiempo de ejecución crece de forma exponencial: con posiciones cercanas a 45 el programa ya tarda varios segundos y con valores mayores a 50 tardaría minutos. También se identificó que unsigned long long solo representa correctamente la serie hasta la posición 93; a partir de la 94 el resultado se desborda, aunque en la práctica la versión recursiva tardaría demasiado en llegar a ese valor.

Calculadora recursiva. La división recursiva perdía los decimales: el método auxiliar dividirRecursivo regresa un int, por lo que 7 / 2 daba 3.0 en lugar de 3.5. También se identificó que la división solo funciona con números positivos: con un divisor negativo la condición dividendo < divisor nunca se cumple y el método se llama a sí mismo sin terminar (StackOverflowError), y con un dividendo negativo regresa 0. Un problema similar ocurre en potencia cuando el exponente es negativo, ya que nunca llega al caso base. Por último, cada llamada recursiva ocupa memoria en la pila, por lo que multiplicar por valores muy grandes (por ejemplo, 5 × 200000) también termina en un desbordamiento de pila.

Codificación de los archivos. Al abrir los archivos .cpp, los acentos y las ñ de los comentarios aparecían como símbolos extraños. Es un problema de codificación, porque el archivo se guardó en un formato y se abrió en otro; no afecta la ejecución, pero se ve mal en el código. Esto ocurrió en los programas de esta práctica y de la siguiente.

SOLUCIÓN DE ERRORES

Factorial. Se agregó en main una validación que muestra un mensaje cuando el valor es menor que cero, ya que el factorial no está definido para esos casos (sin ella, el caso base n <= 1 regresaría 1 sin avisar). Para reducir el desbordamiento se utilizó unsigned long long, el entero estándar de mayor capacidad, con el que el resultado es correcto hasta 20!; para valores mayores se debe limitar la entrada o usar una biblioteca de números grandes. Para la lectura inválida, la corrección consiste en comprobar el resultado de cin (por ejemplo, con cin.fail()) antes de calcular.

Fibonacci. Se definieron los dos casos base (n <= 1, que regresa n) y el paso recursivo como la suma de las dos posiciones anteriores, y se agregó la validación de posiciones negativas. Para evitar la repetición de cálculos, la solución consiste en guardar los resultados ya obtenidos (memoización) o calcular la serie con un ciclo, con lo que el tiempo deja de ser exponencial. Se conservó la versión recursiva porque el objetivo de la práctica es aplicar recursividad, tomando en cuenta que no es la forma más eficiente.

Calculadora recursiva. En la multiplicación se agregó una condición para el multiplicador negativo: se calcula con su valor positivo y se invierte el signo del resultado (return -multiplicar(num1, -num2)). En la división se dejó la validación de cero en el método dividir y el cálculo recursivo en el método auxiliar dividirRecursivo, para no repetir la validación en cada llamada. Para conservar los decimales, la corrección consiste en que el auxiliar

regrese float y trabaje con valores absolutos, ajustando el signo al final de forma parecida a la multiplicación; en potencia se debe validar que el exponente no sea negativo antes de iniciar la recursión. Con valores muy grandes conviene usar la versión con ciclo.

Codificación de los archivos. Los archivos se guardaron en UTF-8 y se corrigieron los comentarios, de modo que los acentos y las ñ se ven correctamente en los códigos mostrados.

CÓDIGO

Factorial (C++)

```
#include <iostream>

// Función recursiva para calcular el factorial
unsigned long long factorial(int n) {
    // Caso base: el factorial de 0 y 1 es 1
    if (n <= 1) {
        return 1;
    }
    // momento recursivo: n * factorial de (n-1)
    return n * factorial(n - 1);
}

int main() {
    int numero;
    std::cout << "Ingresa un numero entero positivo: ";
    std::cin >> numero;

    if (numero < 0) {
        std::cout << "El factorial no esta definido para numeros negativos." << std::endl;
    } else {
        std::cout << "El factorial de " << numero << " es " << factorial(numero) << std::endl;
    }

    return 0;
}
```

Fibonacci (C++)

```
#include <iostream>

// Función recursiva para calcular Fibonacci
unsigned long long fibonacci(int n) {
    // Casos base: si n es 0 devuelve 0, si n es 1 devuelve 1
    if (n <= 1) {
        return n;
    }
    // Paso recursivo: la suma de los dos números anteriores
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int numero;
    std::cout << "Ingresa la posicion en la serie de Fibonacci (entero positivo): ";
    std::cin >> numero;

    if (numero < 0) {
        std::cout << "La posicion no puede ser negativa." << std::endl;
    } else {
        std::cout << "El numero de Fibonacci en la posicion " << numero << " es " <<
    fibonacci(numero) << std::endl;
```

```
    }

    return 0;
}
```

Calculadora recursiva (Java)

```
//calculadora recursiva partiendo de la anterior
public class Main {

    //Clase padre
    static class Calculadora {

        // Suma
        int sumar() {
            return 0;
        }

        int sumar(int num1, int num2) {
            return num1 + num2;
        }

        int sumar(int num1, int num2, int num3) {
            return num1 + num2 + num3;
        }

        // Resta
        int restar() {
            return 0;
        }

        int restar(int num1, int num2) {
            return num1 - num2;
        }

        int restar(int num1, int num2, int num3) {
            return num1 - num2 - num3;
        }

        // Multiplicacion
        int multiplicar() {
            return 0;
        }

        int multiplicar(int num1, int num2) {
            return num1 * num2;
        }

        int multiplicar(int num1, int num2, int num3) {
            return num1 * num2 * num3;
        }

        //  Division
        float dividir() {
            return 0;
        }

        float dividir(float num1, float num2) {
            if (num2 == 0) {
                System.out.println("No se puede dividir entre 0");
                return 0;
            }
```

```
            return num1 / num2;
        }

        float dividir(float num1, float num2, float num3) {
            if (num2 == 0 || num3 == 0) {
                System.out.println("No se puede dividir entre 0");
                return 0;
            }

            return num1 / num2 / num3;
        }
    }

    //Clase hija
    static class CalculadoraNueva extends Calculadora {

        // multiplicacion recursiva (sumas sucesivas)
        // num1 * num2 = num1 + (num1 * (num2 - 1))
        @Override
        int multiplicar(int num1, int num2) {
            // Caso base
            if (num2 == 0) {
                return 0;
            }
            // Si el multiplicador es negativo, se invierte el signo
            if (num2 < 0) {
                return -multiplicar(num1, -num2);
            }
            // Caso recursivo
            return num1 + multiplicar(num1, num2 - 1);
        }

        // Potencia recursiva (multiplicaciones sucesivas)
        // base^exp = base * base^(exp - 1)
        int potencia(int base, int exponente) {
            // Caso base
            if (exponente == 0) {
                return 1;
            }
            // Caso recursivo
            return base * potencia(base, exponente - 1);
        }

        // Division recursiva (restas sucesivas)
        // num1 / num2 = 1 + ((num1 - num2) / num2)
        @Override
        float dividir(float num1, float num2) {
            if (num2 == 0) {
                System.out.println("No se puede dividir entre 0");
                return 0;
            }
            return dividirRecursivo(num1, num2);
        }

        // Metodo auxiliar para no repetir la validacion de division entre 0
        // en cada llamada recursiva
        private int dividirRecursivo(float dividendo, float divisor) {
            // Caso base
            if (dividendo < divisor) {
                return 0;
            }
            // Caso recursivo
            return 1 + dividirRecursivo(dividendo - divisor, divisor);
```

```
        }
    }

    public static void main(String[] args) {

        CalculadoraNueva calculadora = new CalculadoraNueva();

        System.out.println("Multiplicacion recursiva");
        System.out.println("6 * 7 = " + calculadora.multiplicar(6, 7));

        System.out.println("\nPotencia recursiva");
        System.out.println("4 elevado a 6 = " + calculadora.potencia(4, 6));

        System.out.println("\nDivision recursiva");
        System.out.println("45 / 5 = " + calculadora.dividir(45, 5));
    }
}
```

PRÁCTICA 15 · PRIMER DEPARTAMENTAL

Recursividad: fractales (Sierpinski y polvo de Cantor)

ERRORES

Polvo de Cantor. El primer problema fue que el programa no mostraba la evolución del fractal por niveles. Se esperaba que cada línea impresa tuviera un tercio más borrado que la anterior, pero la función hacerCantor ya es recursiva y en la primera llamada borra los tercios centrales de todos los niveles, así que las cuatro líneas impresas después de la línea inicial salían idénticas. Se estaba mezclando un ciclo con la recursividad. También se tuvo en cuenta que la longitud debe ser una potencia de 3 ($81 = 3^4$) para que la división entre tres sea exacta en todos los niveles.

Triángulo de Sierpinski. El programa solo funciona correctamente con tamaños que son potencias de 2: al probar con 30, el dibujo salió incompleto y deformado, porque al dividir entre 2 de forma repetida (30, 15, 7, 3, 1) se pierden posiciones y las partes ya no encajan. Otra dificultad fue la confusión con las coordenadas x y y: la matriz se llena con matriz[x][y], pero al imprimirla se recorre de arriba hacia abajo, por lo que fue necesario ajustar los ciclos de impresión para que el resultado quedara en forma de pirámide.

SOLUCIÓN DE ERRORES

Polvo de Cantor. Para ver la evolución real por niveles, la función debe recibir un parámetro adicional que limite la profundidad de la recursión (nivel actual y nivel máximo), y cada impresión debe llamarla con un nivel más en lugar de repetir la función dentro de un ciclo; sin ese límite, la recursión siempre llega hasta el último nivel. En la versión mostrada se conservó el ciclo por niveles, por lo que las líneas intermedias salen iguales al resultado final, pero el fractal se genera correctamente. La longitud se dejó en 81 (3^4).

Triángulo de Sierpinski. Se fijó el tamaño en 32 (2^5), de modo que al dividir entre 2 siempre se llega exactamente al caso base tamano == 1. Para la impresión se recorrió y desde tamano - 1 hasta 0, se escribieron y espacios iniciales para centrar el triángulo y se limitó cada fila con la condición x + y < tamano, con lo que la matriz se muestra en forma de pirámide.

CÓDIGO

Polvo de Cantor (C++)

```
#include <iostream>
#include <vector>

using namespace std;

// funcion recursiva para borrar el tercio central
void hacerCantor(vector<char>& linea, int inicio, int longitud) {
    if (longitud < 1) return; // Caso base

    int tercio = longitud / 3;

    // borra la parte del medio reemplazandola con espacios
    for (int i = inicio + tercio; i < inicio + 2 * tercio; i++) {
        linea[i] = ' ';
    }

    // Si el tercio es mayor a 0, aplicamos la recursividad a los lados restantes
    if (tercio >= 1) {
        hacerCantor(linea, inicio, tercio);                 // lado izquierdo
        hacerCantor(linea, inicio + 2 * tercio, tercio);    // lado derecho
    }
}

int main() {
    int niveles = 4;
    int longitud = 81; // 3 elevado a la potencia de 4

    //empezamos con una línea sólida de '#'
    vector<char> linea(longitud, '#');

    //muestra la línea inicial
    for (char c : linea) cout << c;
    cout << endl;

    //vamos aplicando el algoritmo nivel por nivel para ver la evolucion
    int tamanoActual = longitud;
    for (int i = 0; i < niveles; i++) {
        // ejecutar un nivel de profundidad
        for(int j = 0; j < longitud; j += tamanoActual) {
            hacerCantor(linea, j, tamanoActual);
        }
        tamanoActual /= 3;

        //imprime el resultado de este nivel
        for (char c : linea) cout << c;
        cout << endl;
    }

    return 0;
}
```

Triángulo de Sierpinski (C++)

```
#include <iostream>
#include <vector>

using namespace std;

// función recursiva que divide el espacio y coloca los asteriscos
void generarSierpinski(vector<vector<char>>& matriz, int x, int y, int tamano) {
    // Caso base: si el tamaño es 1, dibujamos la parte más pequeña del fractal
    if (tamano == 1) {
        matriz[x][y] = '*';
        return;
    }
    int mitad = tamano / 2;

    // Llamadas recursivas para los 3 subtriángulos que forman a un triángulo más grande
    generarSierpinski(matriz, x, y, mitad);                 // Esquina base
    generarSierpinski(matriz, x + mitad, y, mitad);         // Desplazamiento en X
    generarSierpinski(matriz, x, y + mitad, mitad);         // Desplazamiento en Y
}

int main() {
    int tamano = 32;
    //crear una cuadrícula llena de espacios vacíos
    vector<vector<char>> matriz(tamano, vector<char>(tamano, ' '));

    // llamamos a la función recursiva comenzando en la coordenada (0,0)
    generarSierpinski(matriz, 0, 0, tamano);

    // Imprimir el resultado en la consola dándole forma piramidal
    for (int y = tamano - 1; y >= 0; y--) {
        // imprime espacios iniciales para centrar el triángulo
        for (int i = 0; i < y; i++) {
            cout << " ";
        }

        // Recorrer la matriz y mostrar su contenido
        for (int x = 0; x + y < tamano; x++) {
            cout << matriz[x][y] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

SEGUNDO DEPARTAMENTAL

Este departamental reúne las prácticas de tipos de dato abstractos (ADT): pila, cola y lista, tanto en su forma estática como dinámica. Todos los programas están escritos en lenguaje C y se compilaron y probaron sin advertencias.

PRÁCTICA 2 · SEGUNDO DEPARTAMENTAL

Tipos de dato abstractos (ADT)

ERRORES

Durante la elaboración de este programa se presentó un error de compilación al declarar la estructura Nodo. El campo que apuntaba al siguiente elemento se escribió inicialmente como Nodo *sig dentro de la propia estructura, pero en ese punto el compilador todavía no conocía el nombre Nodo, por lo que mostraba el mensaje unknown type name 'Nodo'.

Otro problema fue que los datos y las operaciones estaban mezclados dentro de main, por lo que la pila no podía considerarse un tipo de dato abstracto: no existía una separación clara entre qué información guarda y qué operaciones permite realizar.

SOLUCIÓN DE ERRORES

El error de compilación se resolvió declarando el campo como struct Nodo *sig, ya que la etiqueta struct Nodo sí es reconocida dentro de su propia definición, y después se creó el nombre Nodo mediante typedef. De esta forma se obtuvo una estructura autorreferenciada, es decir, que contiene un apuntador a otro elemento del mismo tipo.

Para el segundo problema se definió la estructura Pila y se separaron las operaciones inicializar y estaVacia en funciones independientes. Así, main solamente utiliza la interfaz del tipo de dato sin necesidad de conocer cómo está construido internamente.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

/* ADT Pila: los datos y las operaciones se definen aparte de la implementacion */
typedef struct Nodo {
    int dato;
    struct Nodo *sig;    /* estructura autorreferenciada */
} Nodo;

typedef struct {
    Nodo *tope;
} Pila;

void inicializar(Pila *p) {
    p->tope = NULL;
}

int estaVacia(Pila *p) {
```

```
    return p->tope == NULL;
}

int main(void) {
    Pila pila;
    inicializar(&pila);
    printf("Pila vacia: %s\n", estaVacia(&pila) ? "si" : "no");
    return 0;
}
```

PRÁCTICA 3 · SEGUNDO DEPARTAMENTAL

Pila estática mediante un arreglo

ERRORES

En la primera versión el tope se inicializó en 0 y se incrementaba después de guardar el dato (datos[tope++]). Esto provocó que no se pudiera distinguir entre una pila vacía y una pila con un elemento, y que la operación pop regresara una posición que no contenía ningún dato válido, mostrando valores basura.

Además, al insertar más elementos que la constante MAX, el programa escribía fuera del arreglo y sobrescribía otras variables, ya que no se comprobaba si la pila estaba llena antes de insertar.

SOLUCIÓN DE ERRORES

Se inicializó el tope en -1 para representar una pila vacía. Antes de insertar se comprueba que tope sea distinto de MAX - 1 y antes de extraer se comprueba que tope sea distinto de -1; el índice solamente se modifica después de validar la operación.

Las funciones push y pop regresan 1 o 0 para que main sepa si la operación pudo realizarse, y con ello el programa muestra el mensaje de pila llena en lugar de escribir fuera del arreglo.

CÓDIGO

```
#include <stdio.h>
#define MAX 5

typedef struct {
    int datos[MAX];
    int tope;
} Pila;

void inicializar(Pila *p) {
    p->tope = -1;
}

int push(Pila *p, int valor) {
    if (p->tope == MAX - 1) return 0;   /* pila llena */
    p->datos[++p->tope] = valor;
    return 1;
}

int pop(Pila *p, int *valor) {
    if (p->tope == -1) return 0;        /* pila vacia */
    *valor = p->datos[p->tope--];
    return 1;
}
```

```
int main(void) {
    Pila p;
    int x;
    inicializar(&p);
    for (int i = 1; i <= 6; i++) {
        if (!push(&p, i * 10)) printf("Pila llena, no se inserto %d\n", i * 10);
    }
    while (pop(&p, &x)) printf("Sale: %d\n", x);
    return 0;
}
```

PRÁCTICA 4 · SEGUNDO DEPARTAMENTAL

Cola estática mediante un arreglo

ERRORES

Al principio los índices frente y final solamente se incrementaban. Después de insertar y eliminar varias veces, final llegaba al valor de MAX y el programa escribía fuera del arreglo, aunque todavía hubiera espacios libres al inicio.

También se presentó un problema al comparar los índices para saber si la cola estaba vacía o llena, ya que ambos estados producían la misma condición y el programa indicaba que la cola estaba vacía cuando en realidad estaba llena.

SOLUCIÓN DE ERRORES

Se utilizó un arreglo circular: los índices se actualizan con la expresión (indice + 1) % MAX, de modo que al llegar al final regresan a la posición 0 y se reutilizan los espacios liberados.

Para distinguir entre cola vacía y cola llena se agregó la variable cantidad: la cola está vacía cuando vale 0 y llena cuando es igual a MAX. Esta variable se incrementa al encolar y se decrementa al desencolar.

CÓDIGO

```
#include <stdio.h>
#define MAX 5

typedef struct {
    int datos[MAX];
    int frente, final, cantidad;
} Cola;

void inicializar(Cola *c) {
    c->frente = 0;
    c->final = -1;
    c->cantidad = 0;
}

int encolar(Cola *c, int valor) {
    if (c->cantidad == MAX) return 0;
    c->final = (c->final + 1) % MAX;
    c->datos[c->final] = valor;
    c->cantidad++;
    return 1;
}
```

```
int desencolar(Cola *c, int *valor) {
    if (c->cantidad == 0) return 0;
    *valor = c->datos[c->frente];
    c->frente = (c->frente + 1) % MAX;
    c->cantidad--;
    return 1;
}

int main(void) {
    Cola c;
    int x;
    inicializar(&c);
    for (int i = 1; i <= 5; i++) encolar(&c, i * 10);
    desencolar(&c, &x);
    desencolar(&c, &x);
    encolar(&c, 60);                    /* reutiliza el espacio liberado */
    encolar(&c, 70);
    while (desencolar(&c, &x)) printf("Sale: %d\n", x);
    return 0;
}
```

PRÁCTICA 5 · SEGUNDO DEPARTAMENTAL

Lista estática mediante un arreglo

ERRORES

Al insertar un elemento en una posición intermedia, el ciclo que desplazaba los datos recorría el arreglo de izquierda a derecha. Esto hizo que el valor de la posición se copiara en todas las siguientes y se perdieran los datos originales, quedando la lista con valores repetidos.

Otro error fue mostrar la lista recorriendo el arreglo hasta MAX en lugar de hasta la cantidad real de elementos, por lo que se imprimían ceros y valores basura después de los datos.

SOLUCIÓN DE ERRORES

El desplazamiento se cambió para recorrer de derecha a izquierda (desde n hasta pos), con lo que cada dato se copia antes de ser sobrescrito. Al eliminar se hace el desplazamiento contrario, de izquierda a derecha, y se decrementa n.

Se agregó la variable n para controlar la cantidad real de elementos, se valida que la posición esté entre 0 y n, y se comprueba que no se exceda la capacidad antes de insertar. Los recorridos se realizan únicamente hasta n.

CÓDIGO

```
#include <stdio.h>
#define MAX 10

typedef struct {
    int datos[MAX];
    int n;
} Lista;

int insertar(Lista *l, int pos, int valor) {
    if (l->n == MAX || pos < 0 || pos > l->n) return 0;
    for (int i = l->n; i > pos; i--)        /* de derecha a izquierda */
```

```
        l->datos[i] = l->datos[i - 1];
    l->datos[pos] = valor;
    l->n++;
    return 1;
}

int eliminar(Lista *l, int pos) {
    if (pos < 0 || pos >= l->n) return 0;
     for (int i = pos; i < l->n - 1; i++)
        l->datos[i] = l->datos[i + 1];
    l->n--;
    return 1;
}

void mostrar(Lista *l) {
    for (int i = 0; i < l->n; i++) printf("%d ", l->datos[i]);
    printf("\n");
}

int main(void) {
    Lista l = {{0}, 0};
    insertar(&l, 0, 15);
    insertar(&l, 1, 25);
    insertar(&l, 1, 20);                    /* inserta en medio */
    mostrar(&l);
    eliminar(&l, 0);
    mostrar(&l);
    return 0;
}
```

PRÁCTICA 6 · SEGUNDO DEPARTAMENTAL

Pila dinámica mediante apuntadores

ERRORES

Al ejecutar la operación pop con la pila vacía, el programa se cerró con un error de segmentación, debido a que se intentaba leer el dato del nodo cuando el tope era NULL.

Además, al revisar el programa con la herramienta AddressSanitizer se detectó una fuga de memoria, porque los nodos creados con malloc no se liberaban al terminar el programa.

SOLUCIÓN DE ERRORES

Se agregó la comprobación de que el tope sea diferente de NULL antes de extraer. En la operación pop se guarda el nodo en un apuntador auxiliar, se mueve el tope al siguiente nodo y después se libera el nodo extraído con free.

También se creó la función liberar, que vacía la pila con pop hasta que no quedan nodos, y se comprobó que malloc no regrese NULL antes de utilizar el nodo reservado.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Nodo {
    int dato;
    struct Nodo *sig;
} Nodo;

typedef struct {
    Nodo *tope;
} Pila;

void inicializar(Pila *p) {
    p->tope = NULL;
}

int push(Pila *p, int valor) {
    Nodo *nuevo = malloc(sizeof(Nodo));
    if (nuevo == NULL) return 0;
    nuevo->dato = valor;
    nuevo->sig = p->tope;
    p->tope = nuevo;
    return 1;
}

int pop(Pila *p, int *valor) {
    if (p->tope == NULL) return 0;
    Nodo *aux = p->tope;
    *valor = aux->dato;
    p->tope = aux->sig;
    free(aux);
    return 1;
}

void liberar(Pila *p) {
    int x;
    while (pop(p, &x)) { }
}

int main(void) {
    Pila p;
    int x;
    inicializar(&p);
    push(&p, 5);
    push(&p, 8);
    push(&p, 13);
    if (pop(&p, &x)) printf("Sale: %d\n", x);
    liberar(&p);
    return 0;
}
```

PRÁCTICA 7 · SEGUNDO DEPARTAMENTAL

Cola dinámica mediante apuntadores

ERRORES

Al eliminar el último elemento de la cola, el apuntador frente quedaba en NULL pero el apuntador final seguía señalando al nodo que ya se había liberado. Al insertar un nuevo elemento, el programa enlazaba el nodo en memoria liberada, por lo que se producía un error de uso de memoria liberada y los datos nuevos no se mostraban correctamente.

SOLUCIÓN DE ERRORES

Se agregó la condición que, después de extraer, comprueba si frente quedó en NULL y, en ese caso, también asigna NULL a final. Al insertar en una cola vacía ambos apuntadores se actualizan para señalar al nuevo nodo; en caso contrario, el último nodo se enlaza con el nuevo.

Para comprobar la solución se probó la cola con cero, uno y varios elementos, prestando atención al caso de eliminar el último elemento y volver a insertar.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Nodo {
    int dato;
    struct Nodo *sig;
} Nodo;

typedef struct {
    Nodo *frente, *final;
} Cola;

void inicializar(Cola *c) {
    c->frente = c->final = NULL;
}

int encolar(Cola *c, int valor) {
    Nodo *nuevo = malloc(sizeof(Nodo));
    if (nuevo == NULL) return 0;
    nuevo->dato = valor;
    nuevo->sig = NULL;
    if (c->final == NULL) c->frente = nuevo;   /* cola vacia */
    else c->final->sig = nuevo;
    c->final = nuevo;
    return 1;
}

int desencolar(Cola *c, int *valor) {
    if (c->frente == NULL) return 0;
    Nodo *aux = c->frente;
    *valor = aux->dato;
    c->frente = aux->sig;
    if (c->frente == NULL) c->final = NULL;    /* se elimino el ultimo */
    free(aux);
    return 1;
}
```

```
int main(void) {
    Cola c;
    int x;
    inicializar(&c);
    encolar(&c, 12);
    desencolar(&c, &x);
    encolar(&c, 24);                          /* prueba con un solo elemento */
    encolar(&c, 36);
    while (desencolar(&c, &x)) printf("Sale: %d\n", x);
    return 0;
}
```

PRÁCTICA 7 BIS · SEGUNDO DEPARTAMENTAL

Lista dinámica mediante apuntadores

ERRORES

La función insertarInicio recibía el apuntador a la cabeza por valor (Nodo *cabeza). Al regresar a main la lista seguía vacía y la función mostrar no imprimía nada, ya que la modificación solo afectaba a una copia del apuntador. Además, cada nodo creado quedaba sin referencia y producía una fuga de memoria.

SOLUCIÓN DE ERRORES

Se cambió el parámetro a un doble apuntador (Nodo **cabeza) y la función se llama enviando la dirección de la lista (&lista). De esta forma la función puede modificar el apuntador original de main.

La función liberar también recibe un doble apuntador, recorre la lista guardando cada nodo en un apuntador auxiliar y lo libera con free, dejando la cabeza en NULL al terminar.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Nodo {
    int dato;
    struct Nodo *sig;
} Nodo;

int insertarInicio(Nodo **cabeza, int valor) {
    Nodo *nuevo = malloc(sizeof(Nodo));
    if (nuevo == NULL) return 0;
    nuevo->dato = valor;
    nuevo->sig = *cabeza;
    *cabeza = nuevo;
    return 1;
}

void mostrar(Nodo *cabeza) {
    for (Nodo *p = cabeza; p != NULL; p = p->sig)
        printf("%d ", p->dato);
    printf("\n");
}

 void liberar(Nodo **cabeza) {
     while (*cabeza != NULL) {
         Nodo *aux = *cabeza;
```

```
        *cabeza = (*cabeza)->sig;
        free(aux);
    }
}

int main(void) {
    Nodo *lista = NULL;
    insertarInicio(&lista, 10);
    insertarInicio(&lista, 20);
    insertarInicio(&lista, 30);
    mostrar(lista);
    liberar(&lista);
    return 0;
}
```

PRÁCTICA 8 · SEGUNDO DEPARTAMENTAL

Pila dinámica con librerías del IDE (stdlib)

ERRORES

El lenguaje C no incluye clases de colecciones como otros lenguajes, por lo que para esta práctica se utilizaron las funciones de la biblioteca estándar stdlib.h (malloc, realloc y free) para crear una pila sobre un arreglo que crece. En la primera versión se escribió p->datos = realloc(p->datos, ...) directamente; si realloc fallaba regresaba NULL y se perdía la dirección del bloque original.

También se omitió multiplicar la nueva capacidad por sizeof(int), por lo que se reservaban menos bytes de los necesarios y el programa escribía fuera del bloque de memoria. Finalmente, al terminar el programa no se liberaba el arreglo.

SOLUCIÓN DE ERRORES

Se utilizó un apuntador auxiliar para recibir el resultado de realloc y solamente se asigna a p->datos cuando no es NULL. El tamaño solicitado se calcula como capacidad * sizeof(int).

La pila duplica su capacidad cuando el tope llega al último espacio y se creó la función liberar, que ejecuta free sobre el arreglo al terminar.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *datos;
    int tope;
    int capacidad;
} Pila;

int inicializar(Pila *p) {
    p->capacidad = 2;
    p->tope = -1;
    p->datos = malloc(p->capacidad * sizeof(int));
    return p->datos != NULL;
}

int push(Pila *p, int valor) {
```

```
    if (p->tope == p->capacidad - 1) {           /* pila llena: crece */
        int nueva = p->capacidad * 2;
        int *aux = realloc(p->datos, nueva * sizeof(int));
        if (aux == NULL) return 0;               /* p->datos sigue siendo valido */
        p->datos = aux;
        p->capacidad = nueva;
    }
    p->datos[++p->tope] = valor;
    return 1;
}

int pop(Pila *p, int *valor) {
    if (p->tope == -1) return 0;
    *valor = p->datos[p->tope--];
    return 1;
}

void liberar(Pila *p) {
    free(p->datos);
    p->datos = NULL;
    p->tope = -1;
    p->capacidad = 0;
}

int main(void) {
    Pila p;
    int x;
    if (!inicializar(&p)) return 1;
    for (int i = 1; i <= 6; i++) push(&p, i * 10);
    printf("Capacidad actual: %d\n", p.capacidad);
    while (pop(&p, &x)) printf("Sale: %d\n", x);
    liberar(&p);
    return 0;
}
```

PRÁCTICA 9 · SEGUNDO DEPARTAMENTAL

Cola dinámica con librerías del IDE (stdlib)

ERRORES

Al hacer crecer la cola con realloc, los elementos que ya habían dado la vuelta en el arreglo circular quedaban separados: el frente estaba en medio del arreglo y parte de los datos al inicio. Al ampliar el arreglo la cola sacaba los datos en un orden incorrecto y aparecían valores sin inicializar.

SOLUCIÓN DE ERRORES

Cuando la cola se llena se reserva un arreglo nuevo con malloc y se copian los elementos en orden, empezando desde el frente y utilizando la expresión (frente + i) % capacidad. Después se libera el arreglo anterior y el frente se reinicia en 0.

Con este cambio se conserva el orden FIFO aunque la cola haya dado la vuelta antes de crecer.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *datos;
    int frente, cantidad, capacidad;
} Cola;

int inicializar(Cola *c) {
    c->capacidad = 2;
    c->frente = 0;
    c->cantidad = 0;
    c->datos = malloc(c->capacidad * sizeof(int));
    return c->datos != NULL;
}

int encolar(Cola *c, int valor) {
    if (c->cantidad == c->capacidad) {            /* cola llena: crece */
        int nueva = c->capacidad * 2;
        int *aux = malloc(nueva * sizeof(int));
        if (aux == NULL) return 0;
        for (int i = 0; i < c->cantidad; i++)    /* copia en orden desde el frente */
            aux[i] = c->datos[(c->frente + i) % c->capacidad];
        free(c->datos);
        c->datos = aux;
        c->frente = 0;
        c->capacidad = nueva;
    }
    c->datos[(c->frente + c->cantidad) % c->capacidad] = valor;
    c->cantidad++;
    return 1;
}

int desencolar(Cola *c, int *valor) {
    if (c->cantidad == 0) return 0;
    *valor = c->datos[c->frente];
    c->frente = (c->frente + 1) % c->capacidad;
    c->cantidad--;
    return 1;
}

void liberar(Cola *c) {
    free(c->datos);
    c->datos = NULL;
}

int main(void) {
    Cola c;
    int x;
    if (!inicializar(&c)) return 1;
    encolar(&c, 10);
    encolar(&c, 20);
    desencolar(&c, &x);                           /* el frente ya no esta en 0 */
    encolar(&c, 30);
    encolar(&c, 40);                              /* provoca el crecimiento */
    encolar(&c, 50);
    while (desencolar(&c, &x)) printf("Sale: %d\n", x);
    liberar(&c);
    return 0;
}
```

PRÁCTICA 10 · SEGUNDO DEPARTAMENTAL

Lista dinámica con librerías del IDE (stdlib)

ERRORES

Al eliminar con una posición inexistente (por ejemplo, la posición 7 en una lista de 4 elementos) el programa desplazaba datos fuera del rango. También se olvidó actualizar la variable capacidad después de usar realloc; como la comparación n == capacidad dejaba de cumplirse cuando n superaba el valor anterior, el programa seguía insertando fuera del bloque reservado.

SOLUCIÓN DE ERRORES

Se validó que la posición esté dentro del rango antes de insertar o eliminar, y la función regresa 0 cuando es inválida para que main muestre un mensaje. Después de cada realloc exitoso se actualiza la capacidad.

La lista crece duplicando su capacidad y se libera con la función liberar al terminar el programa.

CÓDIGO

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *datos;
    int n, capacidad;
} Lista;

int inicializar(Lista *l) {
    l->capacidad = 2;
    l->n = 0;
    l->datos = malloc(l->capacidad * sizeof(int));
    return l->datos != NULL;
}

int insertar(Lista *l, int pos, int valor) {
    if (pos < 0 || pos > l->n) return 0;
    if (l->n == l->capacidad) {
        int *aux = realloc(l->datos, l->capacidad * 2 * sizeof(int));
        if (aux == NULL) return 0;
        l->datos = aux;
        l->capacidad *= 2;
    }
    for (int i = l->n; i > pos; i--) l->datos[i] = l->datos[i - 1];
    l->datos[pos] = valor;
    l->n++;
    return 1;
}

int eliminar(Lista *l, int pos) {
    if (pos < 0 || pos >= l->n) return 0;
    for (int i = pos; i < l->n - 1; i++) l->datos[i] = l->datos[i + 1];
    l->n--;
    return 1;
}

void mostrar(Lista *l) {
```

```
    for (int i = 0; i < l->n; i++) printf("%d ", l->datos[i]);
    printf("\n");
}

void liberar(Lista *l) {
    free(l->datos);
    l->datos = NULL;
    l->n = l->capacidad = 0;
}

int main(void) {
    Lista l;
    if (!inicializar(&l)) return 1;
    insertar(&l, 0, 10);
    insertar(&l, 1, 30);
    insertar(&l, 1, 20);                          /* provoca el crecimiento */
    insertar(&l, 3, 40);
    mostrar(&l);
    if (!eliminar(&l, 7)) printf("Posicion invalida\n");
    eliminar(&l, 0);
    mostrar(&l);
    liberar(&l);
    return 0;
}
```

CONCLUSIONES

En las prácticas de recursividad se comprobó que todo programa recursivo necesita un caso base y que cada llamada debe acercarse a él (n − 1, un tercio, la mitad); de lo contrario la función no termina y se produce un desbordamiento de pila. La recursividad permite escribir soluciones más cortas y cercanas a la definición matemática, como el factorial, la serie de Fibonacci y los fractales de Cantor y Sierpinski, pero no siempre es la mejor opción: Fibonacci repite muchos cálculos y las operaciones con enteros pueden desbordar el tipo de dato sin que el programa lo avise. Por eso es importante probar con valores límite (0, 1, negativos y números grandes) y no suponer que un programa está bien solo porque funciona con un ejemplo.

Las prácticas permiten comparar distintas formas de organizar y almacenar información en C. Las implementaciones estáticas utilizan arreglos de capacidad limitada; las dinámicas con apuntadores enlazan nodos y requieren administrar la memoria; y las versiones con las funciones de la biblioteca estándar hacen crecer un arreglo con realloc. La validación de límites, la comprobación de estructuras vacías, la verificación de fopen y malloc, y la liberación o cierre de recursos ayudan a prevenir errores.